EA2. CREACIÓN DE REPOSITORIO

PROCESAMIENTO FUNCIONAL DE DATOS DEL TRANSPORTE URBANO EN TECNOVALLE

Geiner Andrés Altamiranda Marbello

Ingeniería de Software y Datos

Programación Orientada a Objetos II

Jorge Armando Julio Cruz

06 de Noviembre de 2026

Introducción

¿Cómo saber si una ruta necesita refuerzo a las siete de la mañana? Contar las validaciones del día ofrece una pista, pero también hay que distinguir quién entra, quién sale y cuántas personas coinciden en el recorrido. En el caso de TecnoValle, el sistema recibe entre seis y doce millones de registros diarios y el procesamiento puede tardar hasta dos horas (Institución Universitaria Digital de Antioquia, s. f.).

El proyecto TecnoMovil Data reproduce una jornada con datos simulados. A partir de los mismos registros, el programa calcula seis indicadores de movilidad. Una prueba pequeña permite revisar los cálculos a mano y otra de mayor tamaño permite comprobar si se obtienen los mismos resultados al repartir parte del trabajo entre varios hilos.

Justificación

Una validación puede servir para responder varias preguntas. Por ejemplo, la salida de una pasajera cuenta para medir el movimiento de una estación, pero sumarla como un nuevo abordaje duplicaría el uso de la ruta. Separar estas reglas en funciones pequeñas permite explicar cada cifra y cambiar un cálculo sin alterar los registros que necesitan los demás.

Objetivos

Desarrollar en Java un módulo de análisis diario que aplique programación funcional a los registros simulados del sistema de transporte de TecnoValle.

Calcular los seis indicadores mediante funciones puras, expresiones lambda y operaciones sobre colecciones inmutables. Verificar los resultados con dos tamaños de

muestra y preparar el proyecto, el informe y sus evidencias para su publicación en la rama principal de GitHub.

Desarrollo de la actividad

Datos de una jornada

Cada registro representa una validación. Por ejemplo, el usuario 1 entra a las 07:00 por Norte en la ruta R1. A las 07:12 registra su salida en Centro. Ambos registros contienen el identificador del usuario, la ruta, la estación, la acción y la fecha con hora (timestamp). El simulador también genera el viaje de regreso por la tarde.

Reglas de los seis indicadores

Afluencia por estación. Se filtran las entradas y se cuentan por estación. Si una persona entra dos veces en el día, aporta dos ingresos.

Horas pico. Se agrupan entradas y salidas por la hora en que ocurren. Se muestran todas las horas que comparten el mayor número de validaciones.

Rutas más utilizadas. Se cuenta una entrada por viaje y se comparan los totales por ruta. Si dos rutas empatan en primer lugar, aparecen ambas.

Patrones por usuario. Se reúnen las estaciones de cada usuario en el orden en que aparecen sus registros. Centro puede aparecer dos veces seguidas cuando allí termina la ida y comienza el regreso.

Tiempo promedio. Para cada usuario se ordenan los eventos por fecha y hora. Se mide una entrada seguida de una salida en otra estación de la misma ruta. Se suman las duraciones y se dividen entre los tramos válidos. Así se excluye la espera entre la ida y el regreso.

Sobrecarga. Cada ruta empieza vacía. Una entrada suma un pasajero y una salida resta uno. Se combinan los cambios simultáneos y se conserva el mayor saldo del día. La ruta es crítica cuando ese máximo supera el umbral simulado.

Cómo funciona el código

Funciones puras e inmutabilidad

La función esEntrada recibe un registro y responde verdadero o falso. Con el mismo registro, siempre responde lo mismo. Tampoco escribe en pantalla ni cambia el registro. Esto permite usarla tanto para contar ingresos por estación como para comparar rutas. La impresión de resultados queda en el método main.

Registro es un record de Java. Sus cinco campos se fijan al crearlo y sus valores también son inmutables. Al ordenar los eventos, el programa obtiene otra lista con sorted().toList(). Las funciones devuelven mapas y listas de solo lectura, de modo que calcular los tiempos conserva intacto el orden original que se usa para los patrones.

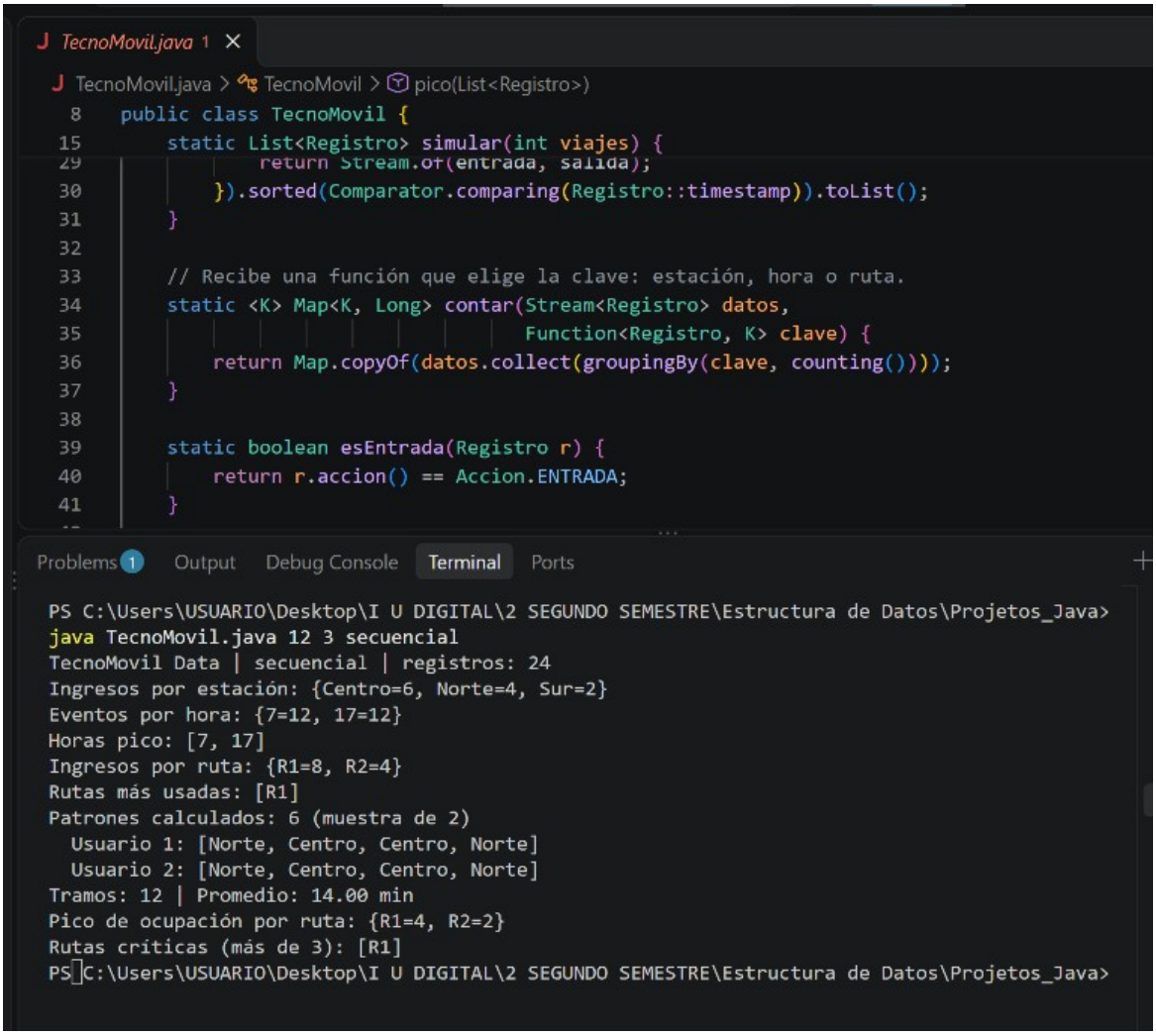
Funciones de orden superior, lambdas y streams

La función contar recibe los datos y otra función que decide cómo agruparlos. Para saber la afluencia, se le pasa Registro::estacion, que extrae la estación. Para hallar las horas pico, se le pasa la lambda r -> r.timestamp().getHour(). La letra r representa un registro y la expresión a la derecha de la flecha devuelve su hora.

Un stream permite encadenar pasos. En afluencia, filter conserva las entradas y groupingBy las reúne por estación. Después, counting cuenta los elementos de cada grupo. Para el promedio, flatMapToLong reúne los segundos de todos los tramos en un solo flujo. Cada tramo aporta una observación, incluso si un usuario realizó más viajes que otro. Para la ocupación, reduce combina bloques ordenados. Si un bloque termina con tres pasajeros y el siguiente alcanza un pico local de dos, el pico de esa segunda parte equivale a cinco. Balance guarda el saldo y el pico en cada paso. Los conteos pueden usar parallelStream() sin compartir un contador entre hilos.

Evidencia de ejecución

Figura 1 *. Resultados de doce viajes con un umbral de tres pasajeros*

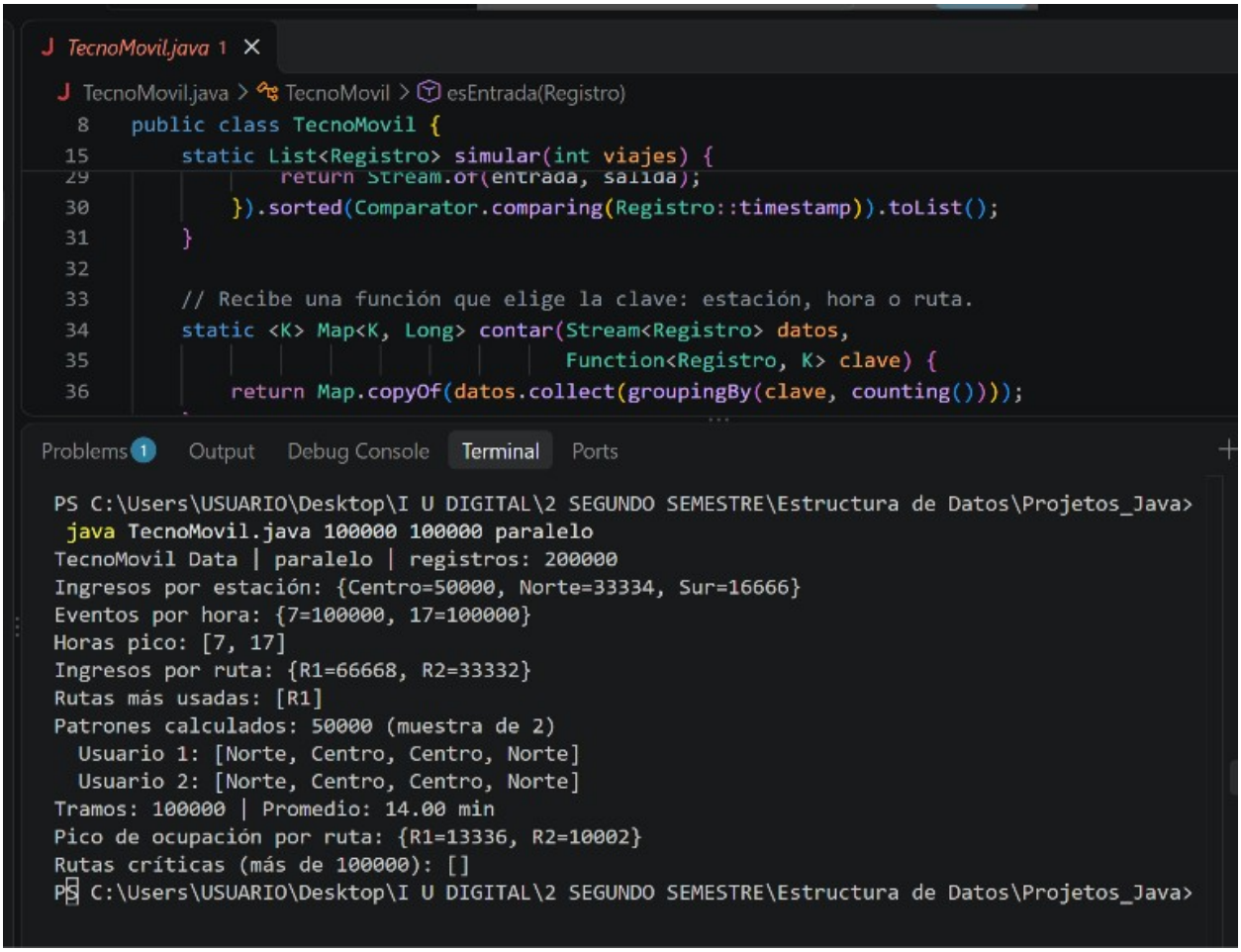


Los doce viajes producen veinticuatro validaciones. En R1 se registran ocho viajes de doce minutos y en R2, cuatro de dieciocho. La suma es 168 minutos. Al dividirla entre doce, se obtiene el promedio de 14 minutos que muestra el programa.

R1 acumula ocho ingresos durante el día, pero su máxima ocupación simultánea es de cuatro pasajeros. Con el umbral en tres, R1 queda marcada como crítica. Al repetir la ejecución con un umbral de cuatro, la lista de rutas críticas queda vacía. Este cambio comprueba la diferencia entre alcanzar el límite y superarlo.

Comprobación con mayor volumen

Figura 2 *Resultados de cien mil viajes en modo paralelo*



La segunda prueba genera 200.000 registros de 50.000 usuarios. Las rutas suman 100.000 ingresos y las dos horas pico reúnen 100.000 validaciones cada una. Se calcularon los patrones de todos los usuarios y se imprimieron dos ejemplos para facilitar la lectura.

La ejecución secuencial con los mismos parámetros produjo los mismos conteos, patrones mostrados, tiempos y alertas. Una prueba adicional con cero viajes devolvió colecciones vacías y el mensaje «sin datos» para el promedio. La prueba de volumen cubre 200.000 registros en memoria. Para evaluar el volumen diario completo del caso haría falta medir tiempo y memoria con varios millones de registros.

Conclusiones

En el ejemplo pequeño, las dos franjas con más actividad comienzan a las 07:00 y a las 17:00. Ambas reúnen doce validaciones. Sin embargo, la decisión de reforzar una ruta depende de la ocupación simultánea. R1 llega a cuatro pasajeros y R2 a dos, aunque ambas operan en las mismas franjas.

Reutilizar la función de conteo permitió responder tres preguntas cambiando únicamente la regla de agrupación o el filtro. Separar las entradas de las salidas también evitó sumar el tiempo que una persona pasa fuera del sistema entre dos viajes.

La prueba con 200.000 registros confirmó que el modo paralelo conserva los resultados observados en el modo secuencial. El código, los comandos de ejecución y las capturas permiten a cada integrante repetir el análisis y comprobar los indicadores antes de entregar el informe.

Repositorios de los integrantes

Repositorio del proyecto (cuenta geiner3894-alt):

https://github.com/geiner3894-alt/EA2-TecnoMovil-Altamiranda-Geiner